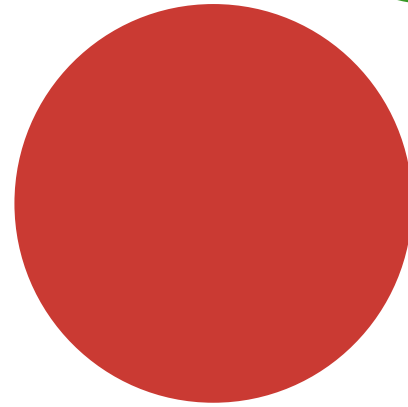




JuliaGeo updates

Anshul Singhvi



JuliaGeo – what is it?

The story of JuliaGeo

Started out as GDAL wrappers and native I/O packages (Shapefile, NetCDF, etc.)

Branched out into wrapping more geospatial libraries (LibGEOS, Proj)

Then started pure Julia algorithms (GeometryOps, AreaInterpolation, etc)

Now a grab bag of geospatial tools, I/O packages, and GeoInterface to tie them together



GeoInterface.jl



What is GeoInterface?

- A common interface defining how to access the contents of a geometry
 - Core functions like `trait`, `getgeom`, `ngeom`, `getpoint`, `npoint`, `crs`, `extent`
 - Allows multiple, very different representations of geometry to coexist in the same ecosystem
- 

I/O interface formalization

- All of the JuliaGeo I/O packages (GeoDataFrames, Shapefile, GeoJSON, GeoParquet, GeoArrow) have similar interfaces
 - But they aren't exactly the same, and sometimes the obvious thing isn't what you need to do
- Working towards a “community interface” – each package should define ``read`` and ``write`` methods, with specific syntax
 - `read(filename; kwargs...)`
 - `write(filename, object; geometrycolumn = nothing, kwargs...)`
 - For write, object may be a table, a vector of geometries, or a single geometry

I/O interface formalization

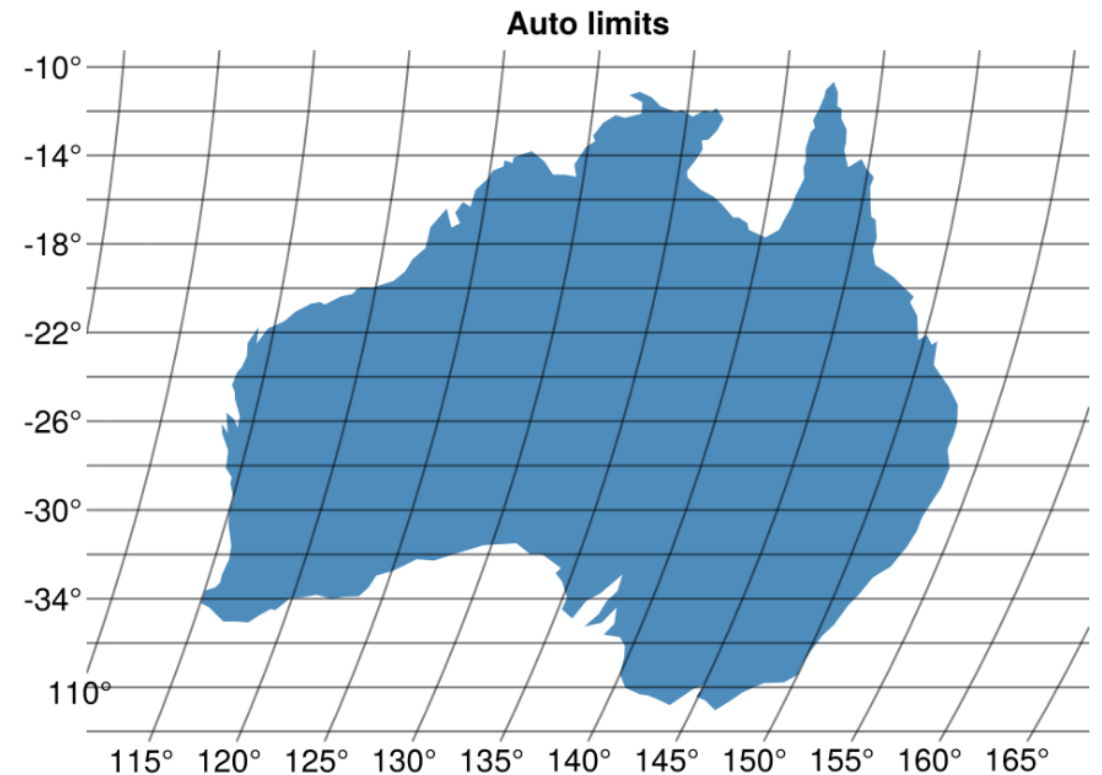
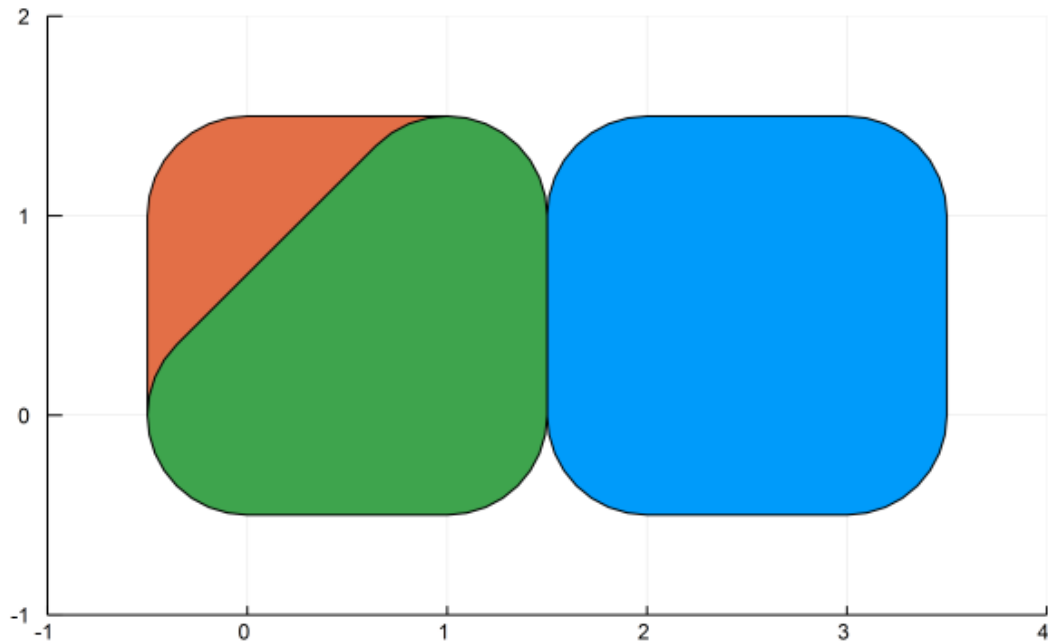
- A single I/O package for all vector data could be quite convenient
- Collect all efforts in GeoDataFrames.jl?
- GeoDataFrames could also introduce spatially accelerated arrays to geometry vectors

Better metadata support

- `crs` and `geometrycolumn` will now be read from `DataAPI.jl` metadata if not overloaded
- This allows arbitrary table types, like `DataFrames`, to persist this information
- When converting e.g Shapefile or a GDAL dataset to a `DataFrame`, this information is not lost!

Plotting support

- With `GeoInterfaceRecipes` (Plots.jl) and `GeoInterfaceMakie`, ``plot(geometry)`` now just works for most `GeoInterface` geometries

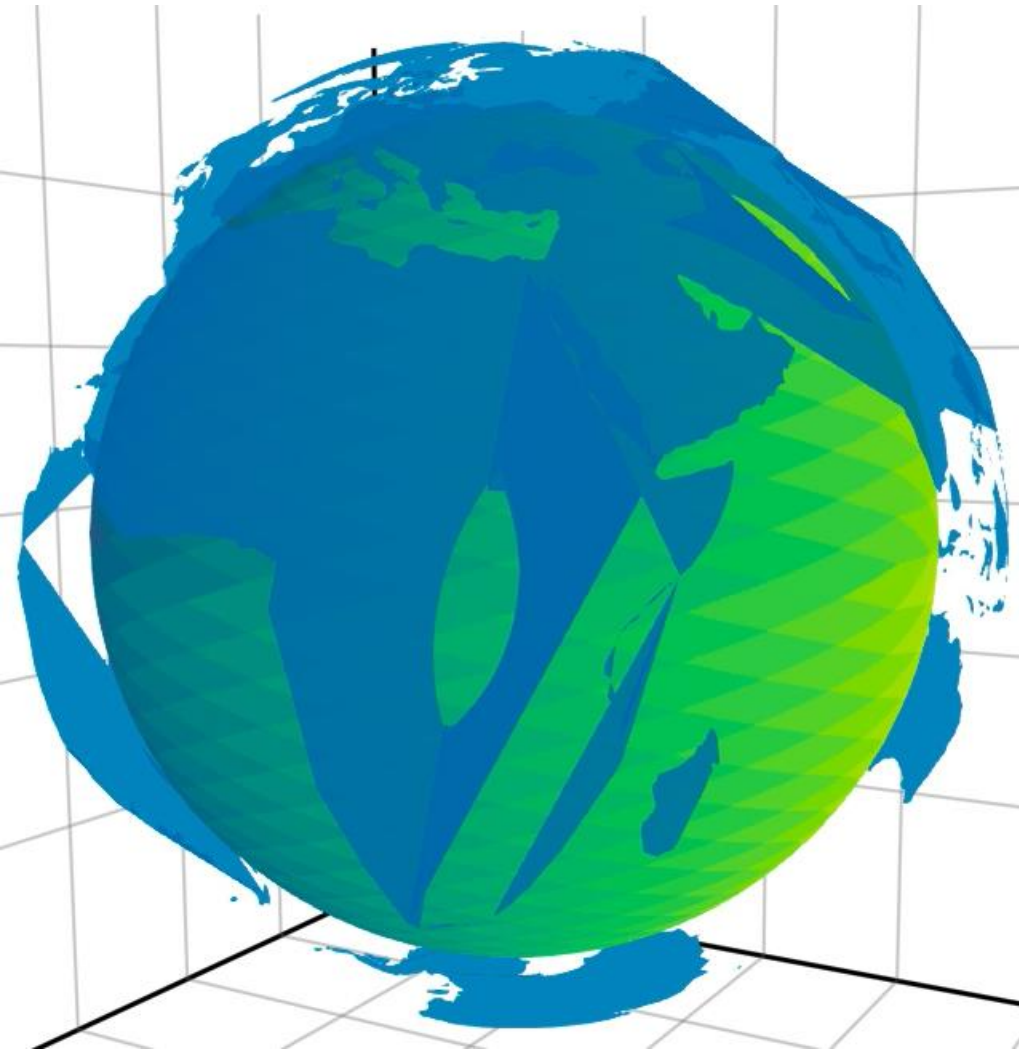




GeometryOps.jl

What is GeometryOps?

- Spatial operations on 2D geometry
- A grab bag of various operations (line and polygon clipping, geometric predicates, arclength computation)
- Core functionality: ways to decompose and iterate over geometry, or collections of geometry
 - `apply(f, target, ...)`: apply some transformation to some “level” of geometry, then reconstruct
 - `applyreduce(f, op, target, ...)`: decompose all inputs to some “level”, then apply `f` and reduce the results with `op`
 - Example: area on multiple geometries decomposes them into single polygons, computes area, then adds it to the area of the next polygon!

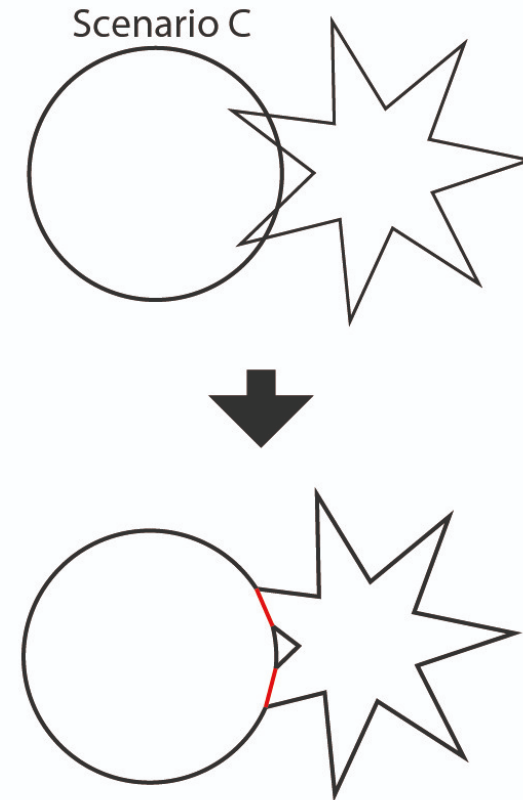
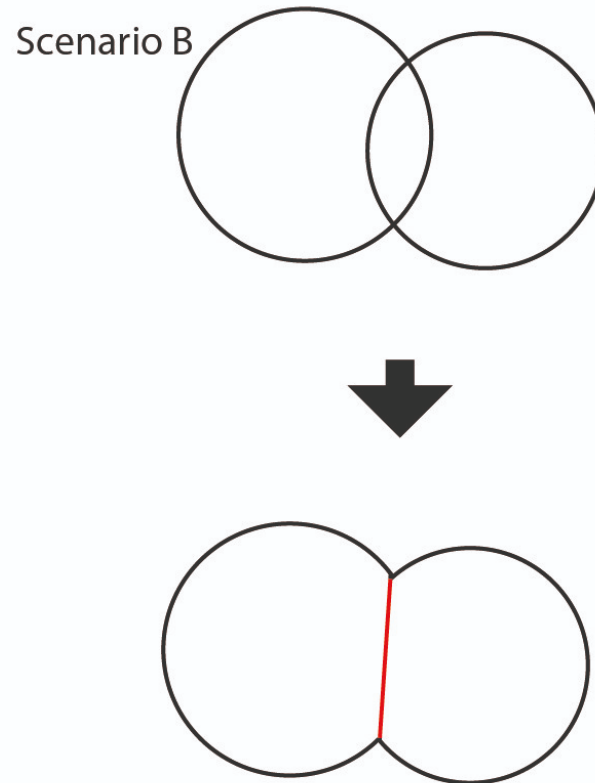
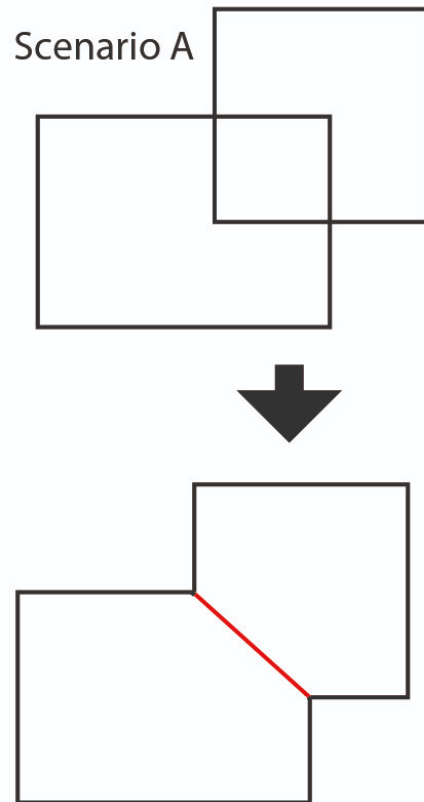


Spherical geometry support (on the way)

- Distributed across multiple PRs, we are getting support for spherical geometry computations
- Things like distance, arclength, etc are easy
- Spherical triangulation and refinement now WIP in `DelaunayTriangulation.jl`
 - `GeometryOps` will both use this internally and provide interfaces
- Spherical-only constructs (spherical caps, etc)

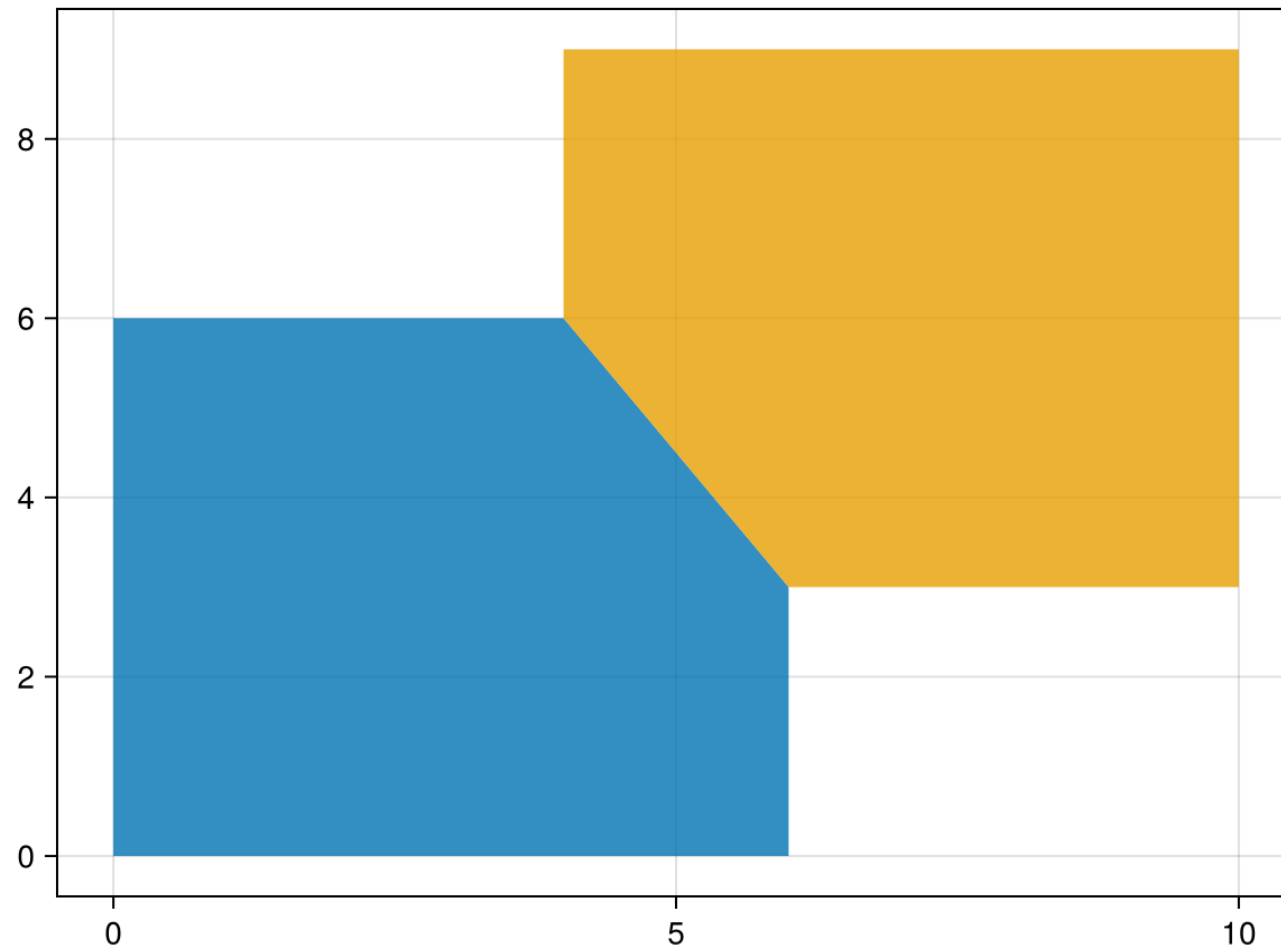
Clipping API

- Clipping – what happens when lines or rings intersect? Where and how do they intersect?
- GeometryOps has a nice internal API that we will make public



Clipping API

- These tools allow



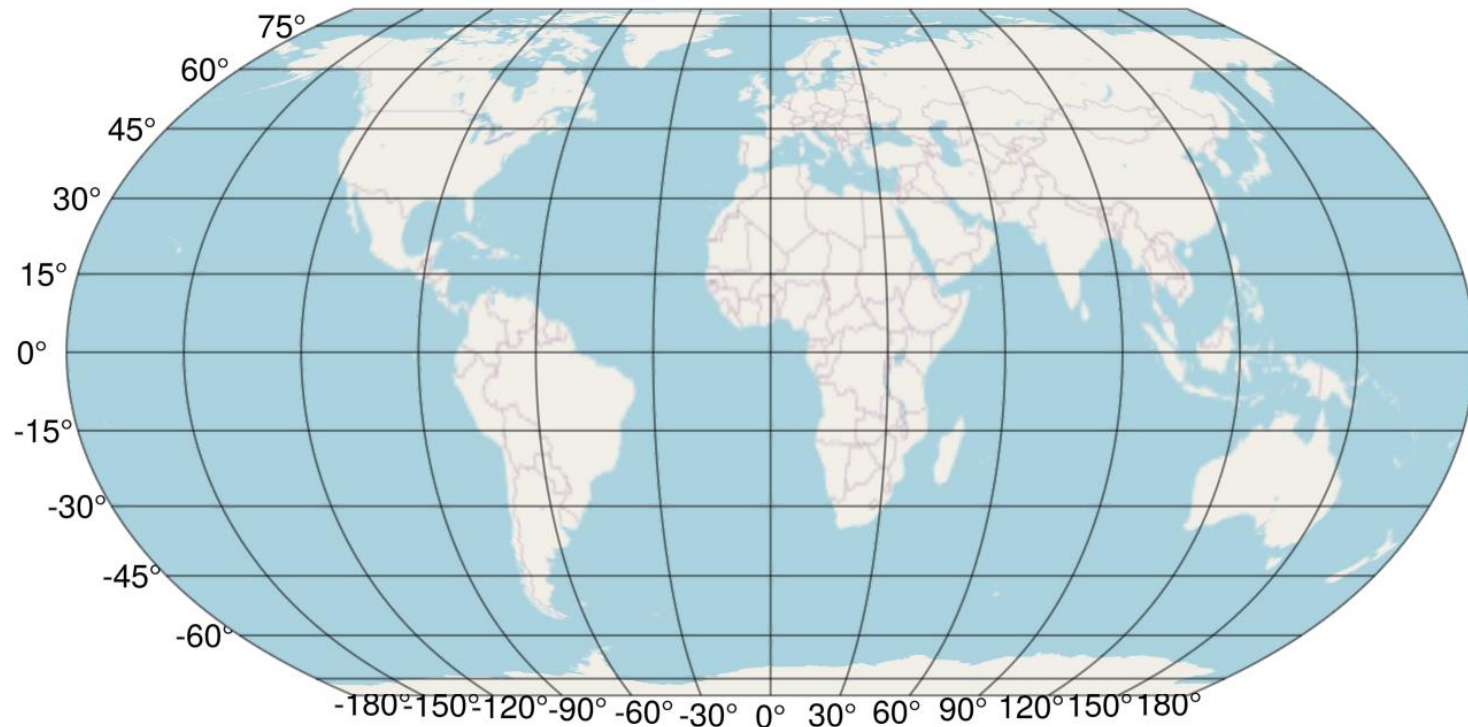


Tyler.jl

GeoMakie integration

Plot tiles in any projection!

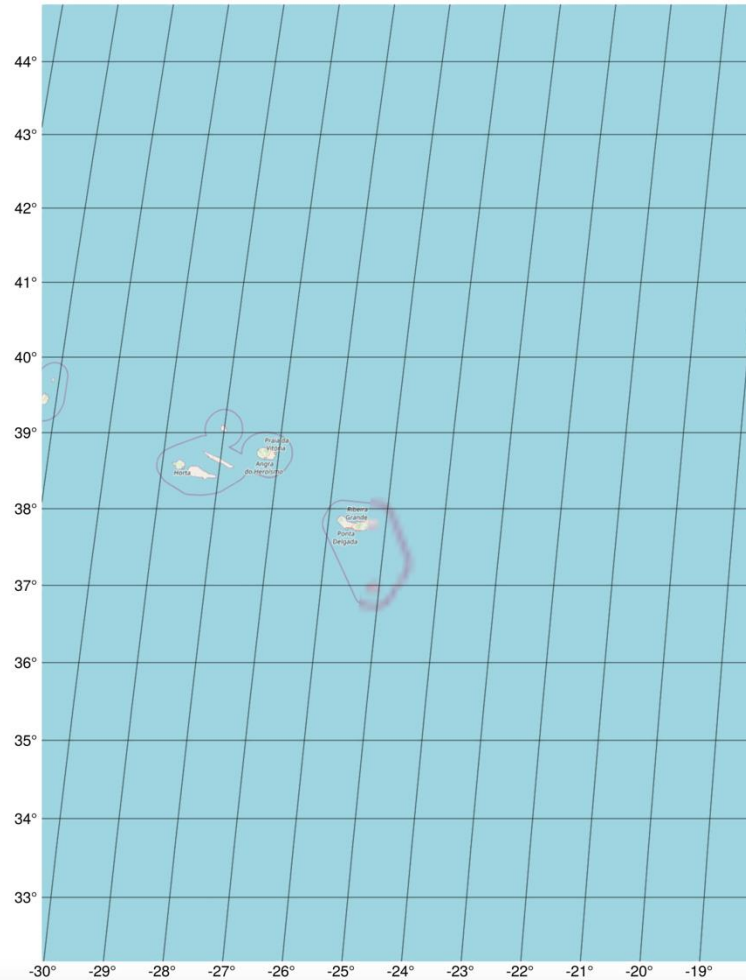
```
Tyler.Map(bbox; axis = (; type = GeoAxis))
```



GeoMakie integration

Plot tiles in any projection!

```
Tyler.Map(bbox; axis = (; type = GeoAxis))
```



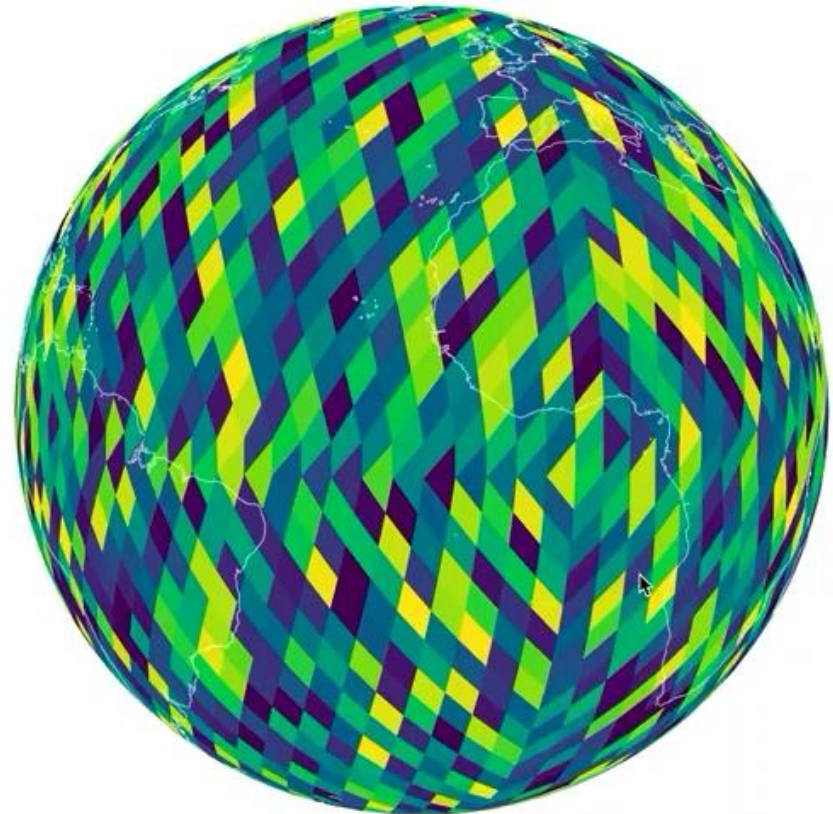


GeoMakie.jl

GlobeAxis

Plot objects on the whole globe – this supports almost any plot type!

- Google Maps-like sphere data explorer
- Should be able to transition to another projection when you zoom in far enough
- Good camera controls and the ability to layer plots
- Best of all – extensibility! Any plot recipe will “just work” and automatically transform to spherical coordinates





GeoAxis improvements

- `reset_limits` kwarg to stop plots from updating limits
`surface!(ax, data;
reset_limits = false)`
- Tighter limits on heatmap and surface-like plots, to decrease extra whitespace
- Better protrusion detection for tighter layouts

Documentation improvements

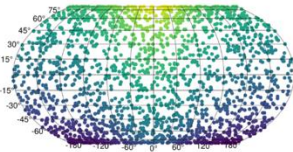
A new "card" system for documentation examples makes it easy to add new examples

Example gallery

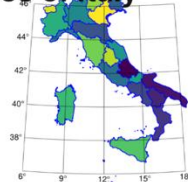
This page is a gallery of various examples.

Basic examples

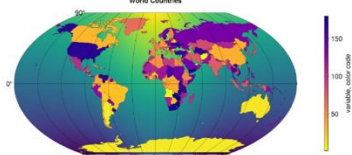
Basic examples



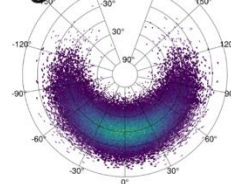
States of Italy



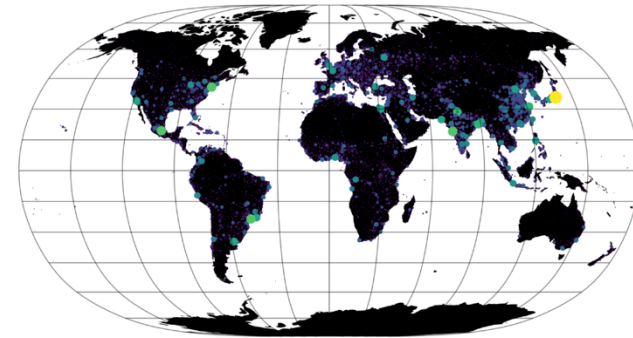
Field and countries



Histogram



World population



This example was contributed by Martijn Visser (@visr)

```
using Makie, CairoMakie, GeoMakie

using GeoMakie: GeoJSON
using GeometryBasics
using Downloads

source = "+proj=longlat +datum=WGS84"
dest = "+proj=natearth2"

land = GeoMakie.assetpath("ne_110m_land.geojson")
land_geo = GeoJSON.read(read(land, String))
pop = GeoMakie.assetpath("ne_10m_populated_places_simple.geojson")
pop_geo = GeoJSON.read(read(pop, String))

begin
```

julia

Documentation improvements

A new "card" system for documentation examples makes it easy to add new examples – either as Markdown files or Literate.jl examples!

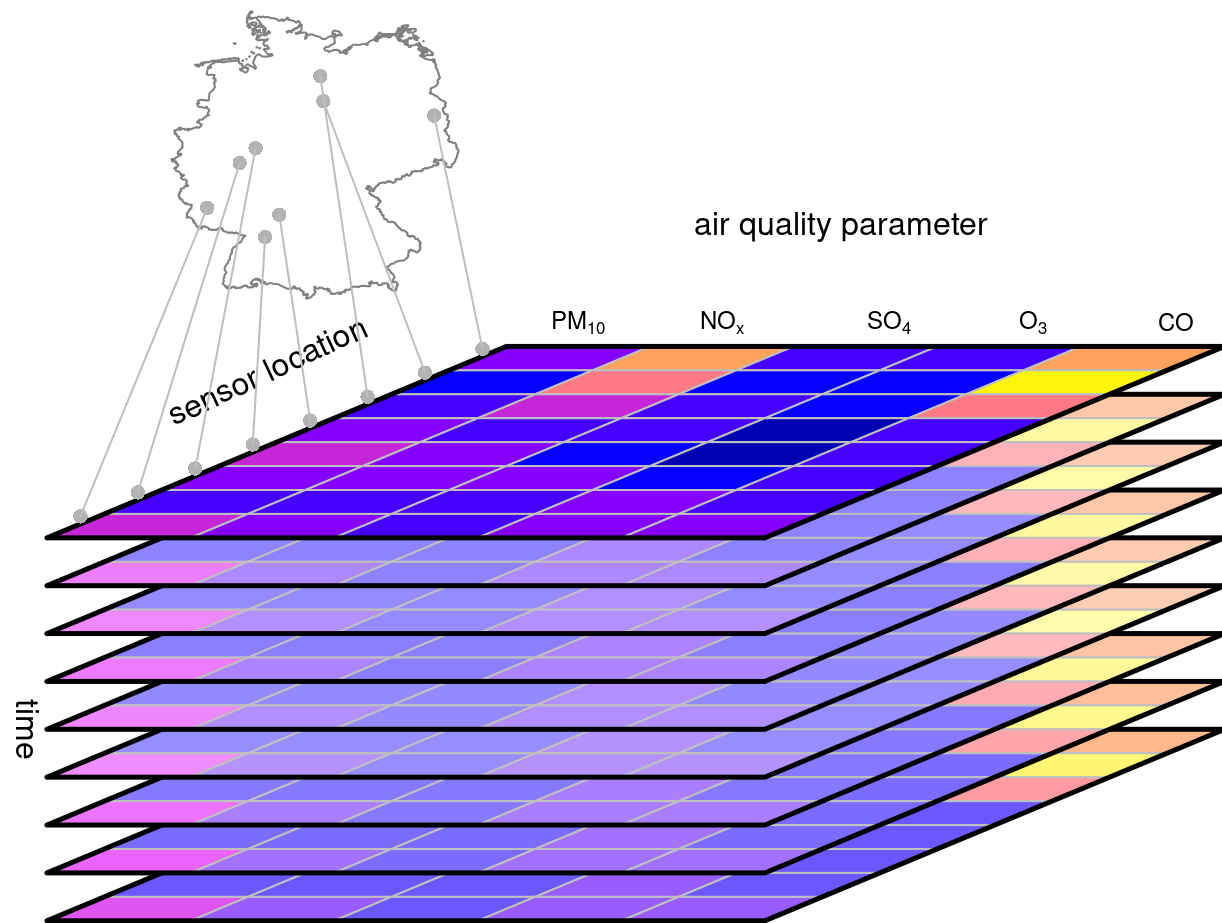
```
poly!(ga, GeoMakie.to_multipoly(geoger.geometry); strokewidth = 0.7
poly!(ga, GeoMakie.to_multipoly(lakes.geometry); strokewidth = 0.7,
fig
#=
```@cardmeta
Title = "German lakes"
Description = "A map of lakes and states in Germany"
Cover = fig
```
=#
```




Vector data cubes

(both types)

Vector data in axes

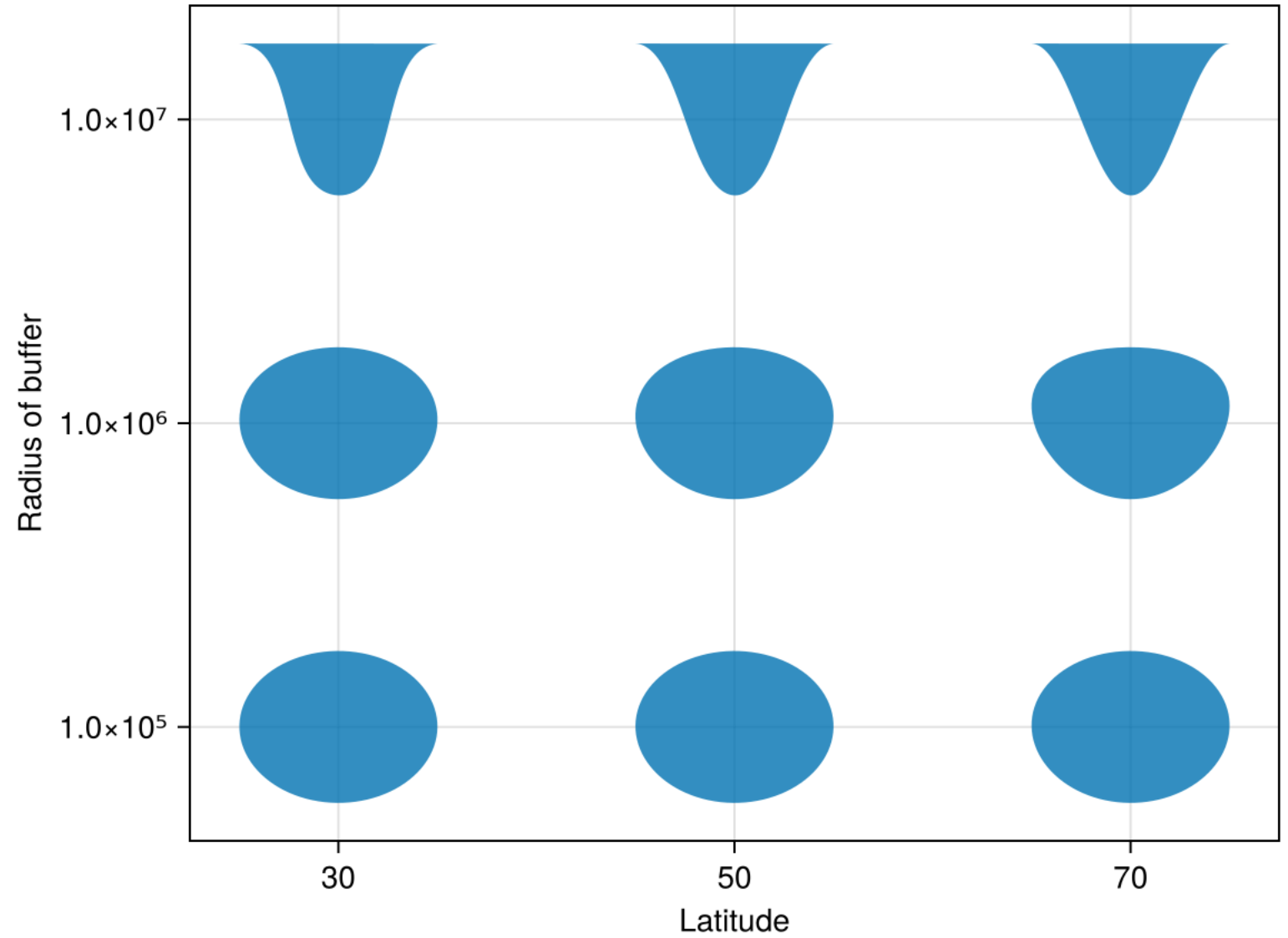




Vector data in axes

- Vector data can be polygons, lines, or points
 - The other axes may be altitude, time, etc
 - Real-world examples:
 - annual glacier elevation profiles,
 - weather station data,
 - political level data (e.g county-level population)
 - Elevates the “feature collection” to n-dims
- 

Geometry as data



Geometry as data

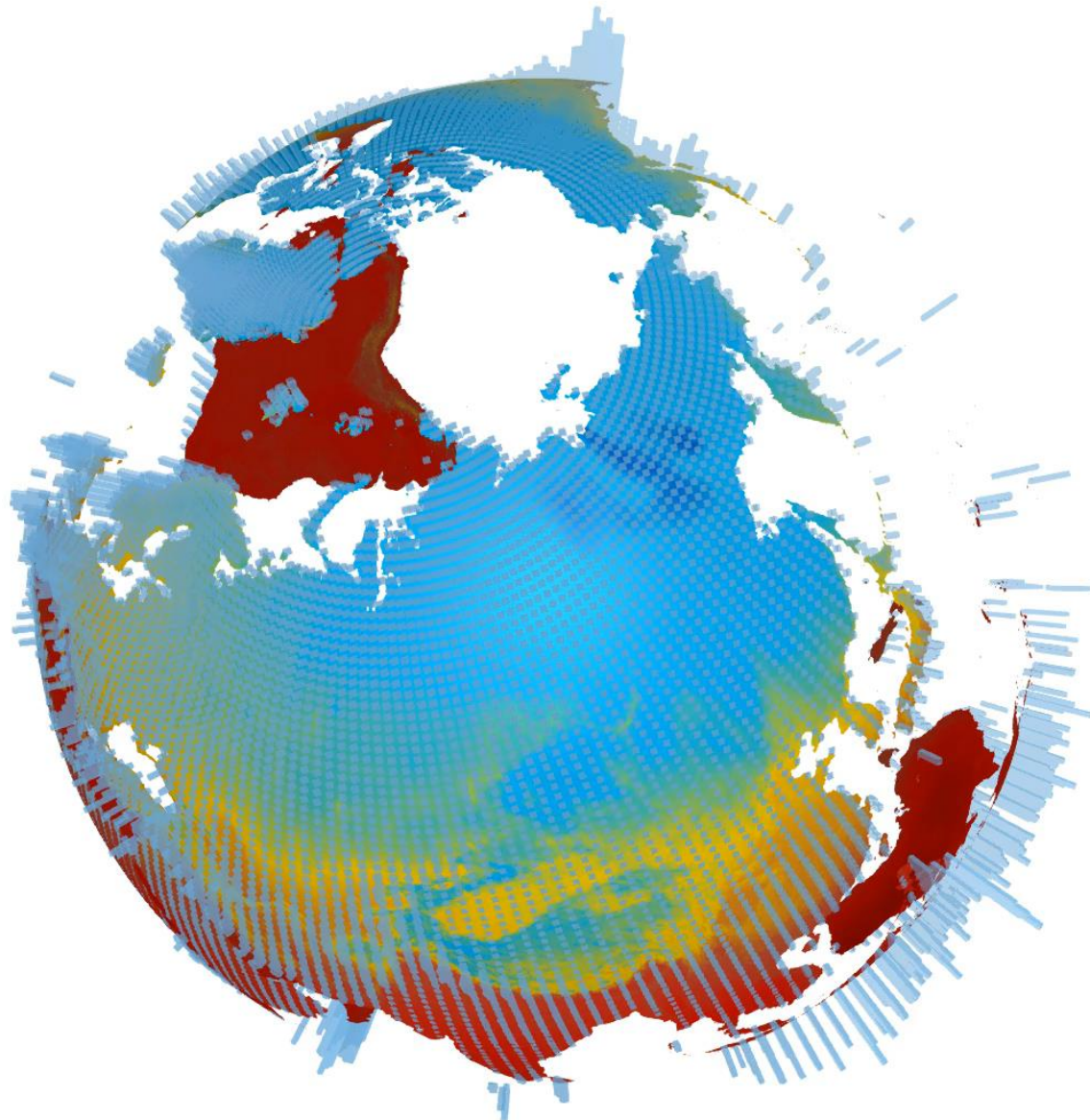
- Consider something like a collection of all the flight routes of an airline
 - One dimension is the route (e.g. LIS-TER)
 - One dimension is the time of the flight
 - The data is the actual path taken, with other layers (?) for weather, delay, etc



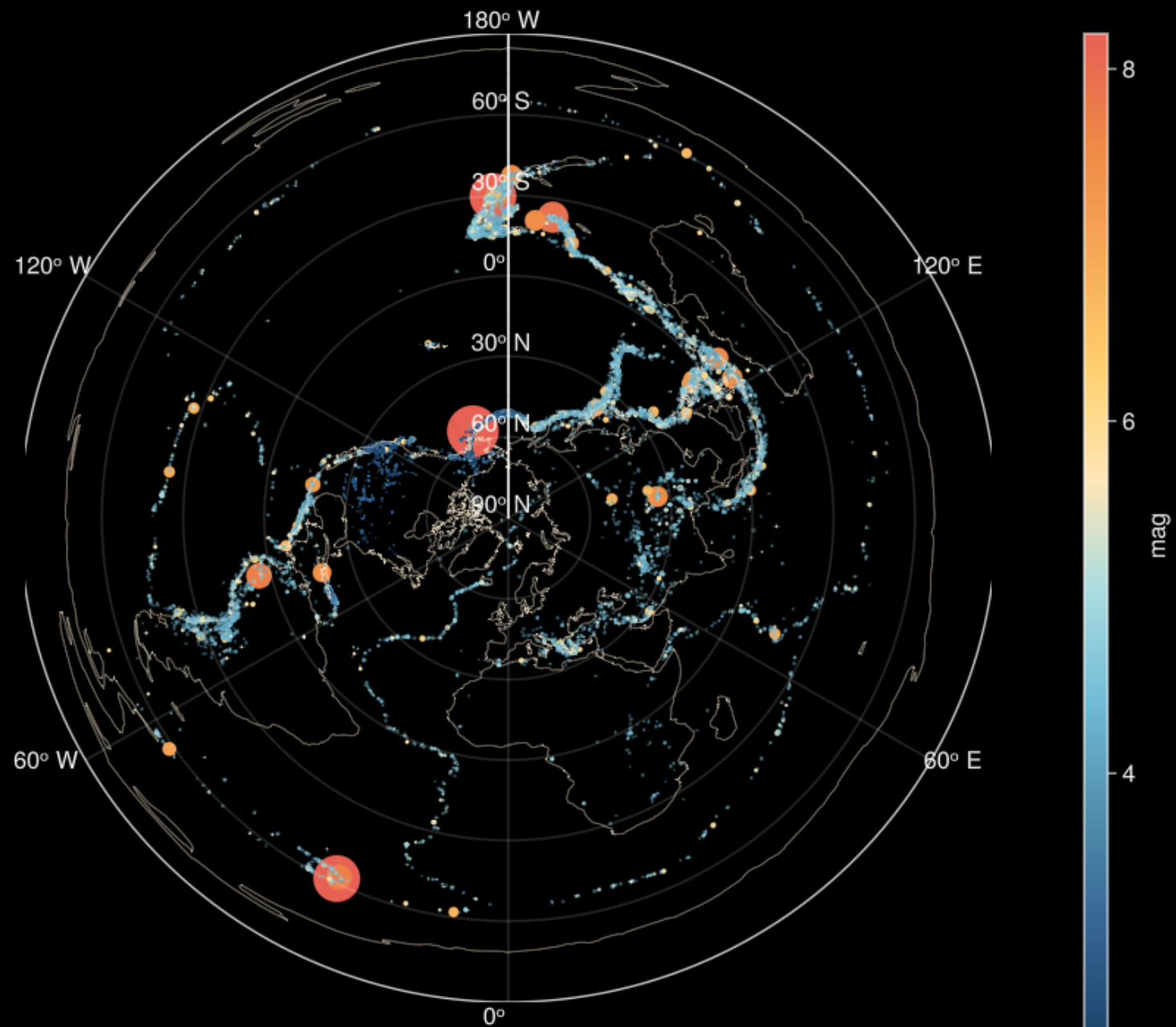
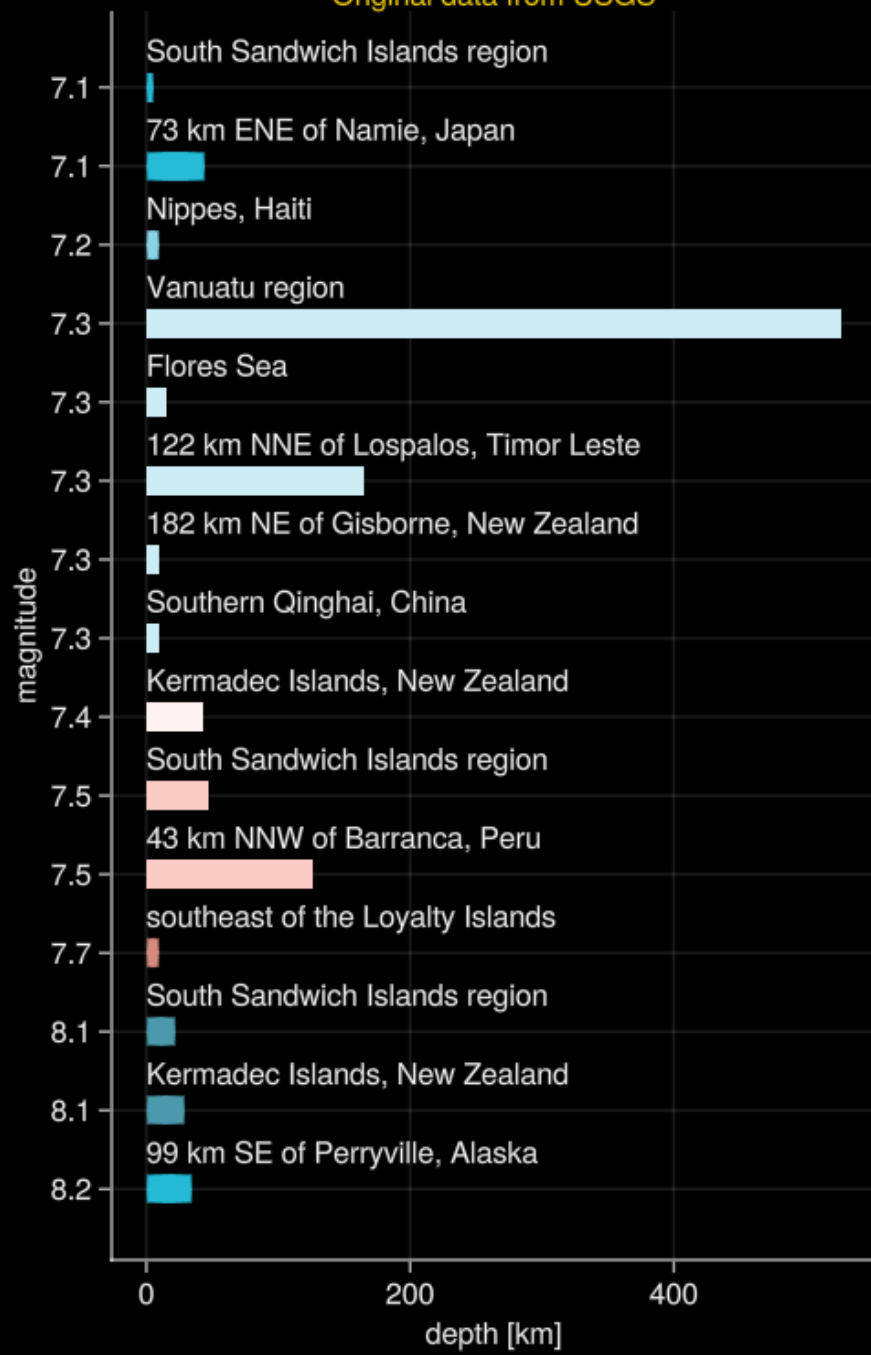


Cool examples

1.00



Earthquakes on Earth between January 2021 and January 2022.
Original data from USGS

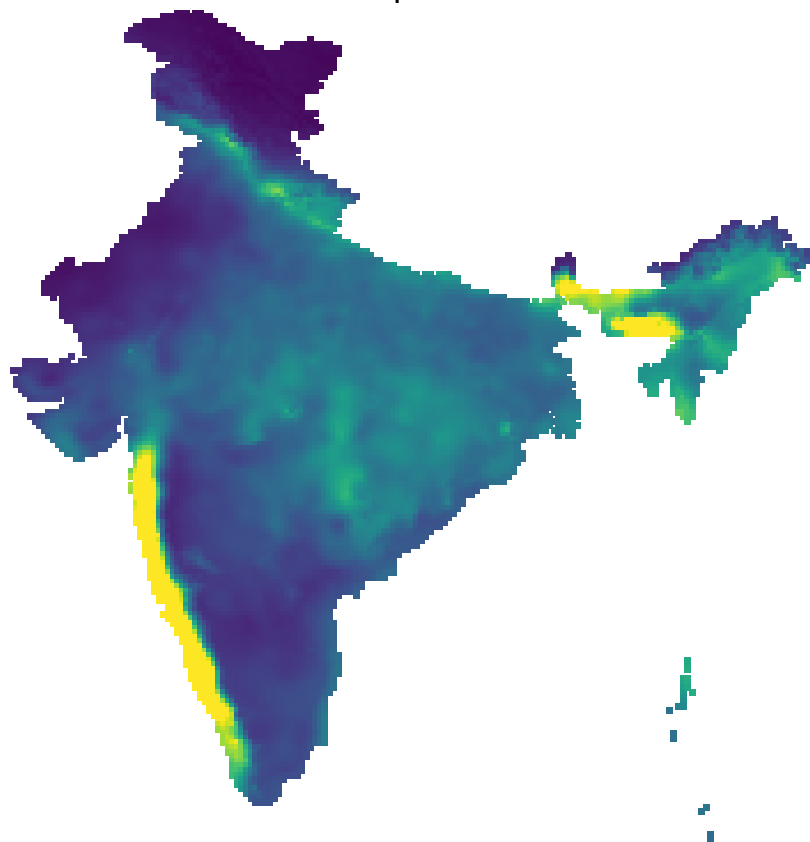


Visualization by @LazarusAlon
using Makie

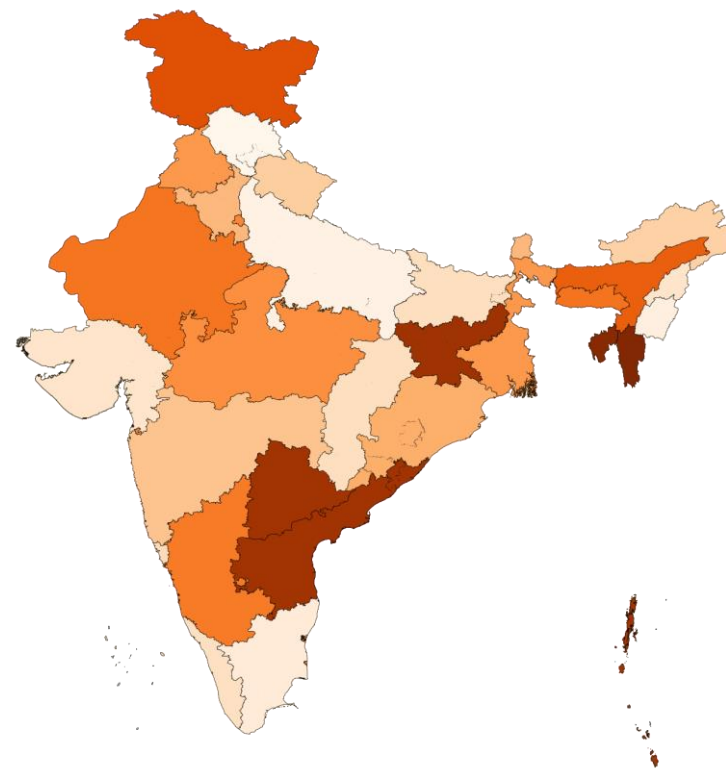
{X}

Does precipitation impact population?

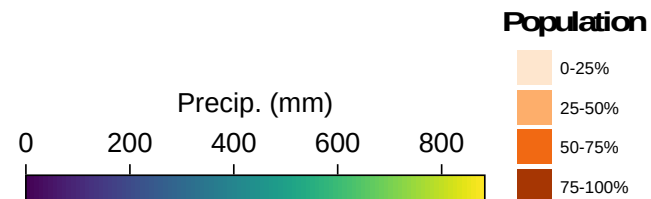
Precipitation



Population (fake)



This answers all of your burning questions - by raining on them!





Live demo